

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное автономное образовательное учреждение высшего образования
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ ТОМСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»



Инженерная школа информационных технологий и робототехники

Отделение информационных технологий

09.04.01 «Информатика и вычислительная техника»

Курсовой проект
по дисциплине «Нейронные сети» на тему
«Реализация Telegram-бота с использованием большой языковой модели
Saiga»

Исполнитель:

студент группы

8BM22

Ямкин Н.Н.

Проверил:

Доцент ОИТ,
ИШИТР

Калиновский И.А.

ОГЛАВЛЕНИЕ

ОГЛАВЛЕНИЕ	2
ВВЕДЕНИЕ	3
ТЕОРЕТИЧЕСКАЯ ЧАСТЬ	4
Архитектура больших языковых моделей.....	4
Общая информация.....	4
ПРАКТИЧЕСКАЯ ЧАСТЬ	7
Создание Telegram бота и настройка соединения с localhost.....	9
Реализация веб-сервиса	11
Память	13
Системный промпт и память	17
Агент Google-поиска	19
ЗАКЛЮЧЕНИЕ	23
СПИСОК ИСТОЧНИКОВ.....	24

ВВЕДЕНИЕ

В эпоху стремительного развития цифровых технологий разработки в области искусственного интеллекта и их дальнейшее внедрение приобретает все большие масштабы. Одним из наиболее востребованных направлений в области искусственного интеллекта последнее время становятся большие языковые модели (Large Language Model, LLM).

LLM активно развиваются и привлекают все больше внимания общественности благодаря своей впечатляющей способности генерировать смысловые и информативные ответы на пользовательские запросы (prompt). Эти системы основаны на комплексной комбинации простых алгоритмов, обширных наборов данных и мощных вычислительных ресурсов. При обучении LLM пытается предсказать следующее слово, а затем обновляет свои параметры на основе правильно или неправильно угаданных слов. Этот подход позволяет модели улучшать свои предсказательные способности и снижать ошибки при генерации текста в будущем [1].

Большие языковые модели, LLM, эффективно дополняют специалистов в их профессиональных обязанностях, освобождая время для других задач, позволяя перестроить бизнес-процессы и создавая новую ценность и для компаний, и для их сотрудников и клиентов [2].

Цель работы: реализация Telegram-бота с использованием русскоязычной LLM Saiga.

Задачи:

1. Исследовать архитектуру LLM;
2. Создать Telegram бота и настроить соединение с локальным веб-сервером;
3. Реализовать веб-сервер для обработки сообщений пользователя с использованием фреймворка Flask;
4. Исследовать возможности библиотеки langchain;
5. Протестировать работу бота и сделать выводы.

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

Архитектура больших языковых моделей

Общая информация

Большая языковая модель — это тип модели глубокого обучения, которая понимает и генерирует текст на человеческом языке. Эти модели обучаются на огромных объемах текстовых данных (книги, статьи, сайты и др. источники) и содержат в себе большое число параметров.

Параметры — это переменные, присутствующие в модели, и которые изменяются в процессе обучения. Считается, что языковая модель является большой если содержит больше одного миллиарда параметров. Именно благодаря большому числу параметров LLM и способны распознавать, переводить, прогнозировать или генерировать текст или другой контент.

Обычно на вход таким моделям подается одно или несколько предложений. По которым модель пытается понять, что от нее хотят и генерирует ответ.

Под капотом у современных LLM-моделей используется архитектура трансформер [3].

Существует множество типов больших языковых моделей, но основные – BERT-like (модели на основе кодировщика Transformer) и GPT-like (модели на основе полного трансформера, либо его кодировщика) модели.

Первый тип моделей применяется для векторизации, классификации, разметки последовательностей, QA (Question Answering), NER (Named Entity Recognition) и т.д., в кросс / мультязычных постановках.



Рисунок 1 – Архитектура BERT

Второй тип – GPT-like модели. Сфера применений генеративных (GPT-like) моделей обширна: к задачам из предыдущего раздела добавляются ведение диалога, машинный перевод, логический и математический вывод, анализ и генерация кода, да и вообще всё, что можно свести к генерации текста по тексту. Наиболее крупные и «умные» модели обычно создаются именно на основе архитектуры декодировщика [4].

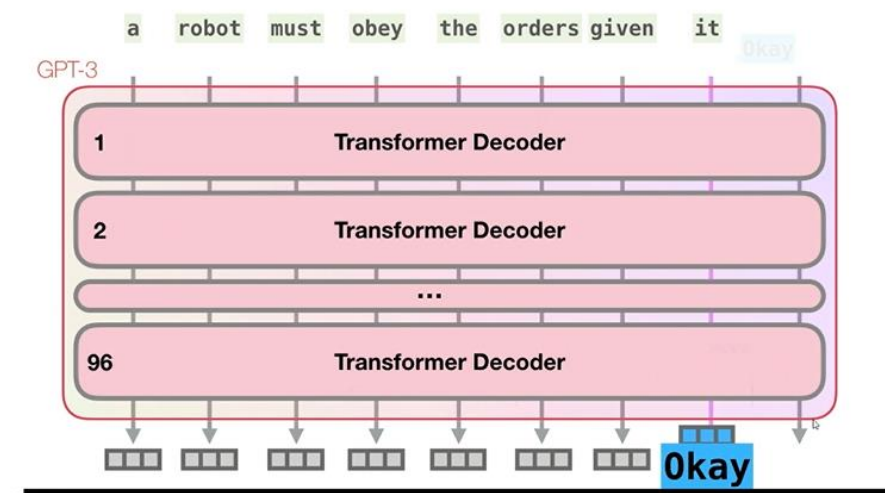


Рисунок 2 – Архитектура GPT-3

В качестве модели мы будем использовать адаптивные веса русскоязычной генеративной модели Saiga2 13B GGUF Q4_K, которые натренировал Илья Гусев.

Сайга-2 — это разговорная языковая модель, которая основана на LLaMA-2 и дообучена на корпусах, сгенерированных ChatGPT, таких как ru_turbo_alpaca, ru_turbo_saiga и gpt_roleplay_realm.

При наличии достаточно мощной GPU адаптивные веса, и саму модель можно скачать, запустить у себя локально и пользоваться offline [5].

ПРАКТИЧЕСКАЯ ЧАСТЬ

Стек технологий и API:

- Python — язык программирования,
- Flask — фреймворк для создания веб-приложений,
- Telegram Bot API,
- Ngrok — сервис для создания туннеля к localhost;
- Langchain – фреймворк, предназначенный для упрощения создания приложений с использованием больших языковых моделей.

Принцип работы бота:

1. Пользователь пишет сообщение телеграмм боту.
2. Telegram пересылает сообщение пользователя на сервер Flask.
3. На сервере LLM Saiga обрабатывает сообщение пользователя и генерирует ответ.
4. Сервер отправляет ответ LLM в Telegram.
5. Пользователь получает ответ LLM.



Рисунок 3 – Схема работы бота

В качестве механизма оповещения о событиях в боте выберем Webhook, т.к. он в десятки раз эффективнее и быстрее Polling (меньше нагружает сеть и процессор), а сам бот работает быстрее и может обрабатывать больше запросов.

Недостатком механизма оповещения событий Webhook является потребность в «белом» IP-адресе (нужно поднимать сервер в облаке, либо использовать сетевые туннели до серверов Telegram).



Рисунок 4 – Сравнение Webhook и Polling [6]

Создание Telegram бота и настройка соединения с локальным сервером

На этом этапе нужно создать бота и получить к нему доступы. Для этого запускаем бота @botfather в Telegram командой `/start` и создаем своего, согласно инструкциям из сообщений @botfather.

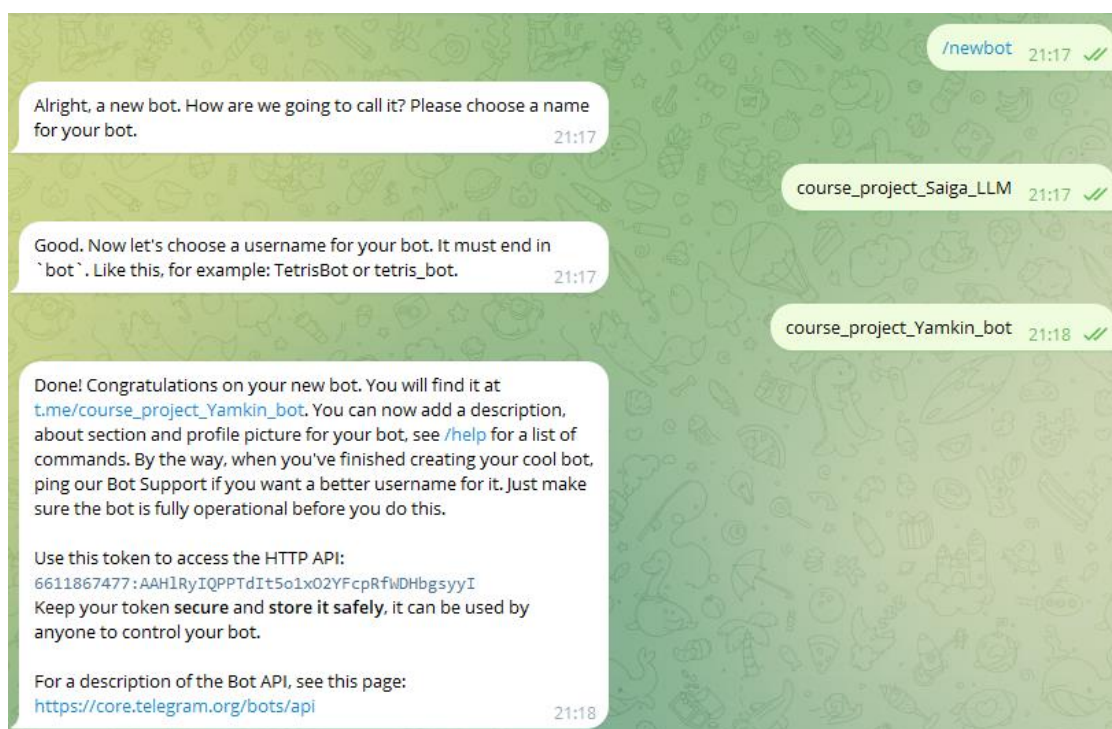


Рисунок 5 – Взаимодействие с @botfather

Бот создан, но, если ему написать какое-нибудь сообщение, он никак на него не отреагирует.

Создадим туннель к localhost с помощью сервиса ngrok.

Ngrok — это сервис, позволяющий создавать туннели на локальный компьютер пользователя [7].

1. Зарегистрируемся на сайте ngrok и выполним установку по инструкции (<https://dashboard.ngrok.com/get-started/setup/linux>).

2. Запустим HTTP туннель на 5015 порте с помощью команды терминала ниже.

```
./ngrok http 5015
```

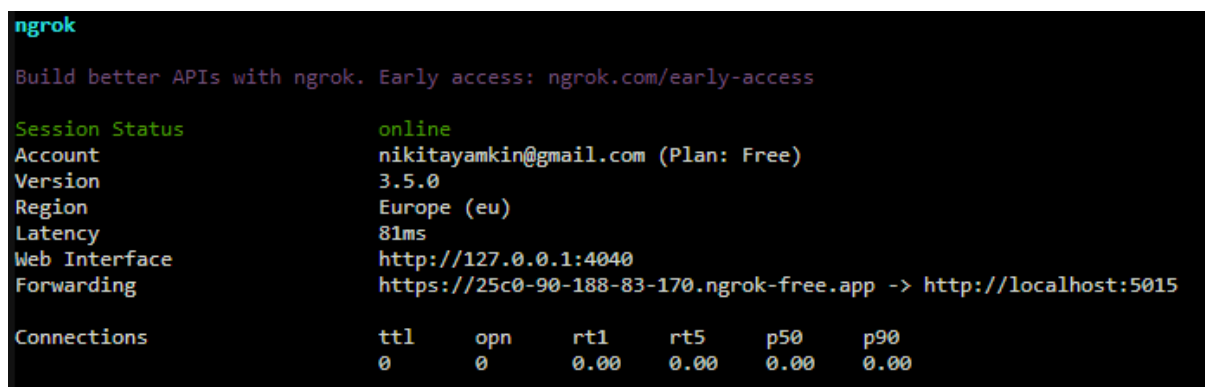


Рисунок 6 – Результат создания туннеля с помощью ngrok

Для того, чтобы Telegram пересылал сообщения на наш сервер, нужно сообщить ему адрес. У нас уже создан туннель, поэтому передадим адрес туннеля в Telegram. Это делается с помощью метода POST setWebhook [7].

```
curl --location --request POST
'https://api.telegram.org/bot6611867477:AAHlRyIQPPTdIt5o1xO2YFcRfWDHbgsyyI/setWebhook' \
--header 'Content-Type: application/json' \
--data-raw '{"url": "https://25c0-90-188-83-170.ngrok-free.app"}'
```

где {6611867477:AAHlRyIQPPTdIt5o1xO2YFcRfWDHbgsyyI} — токен, который нам прислал BotFather, {https://25c0-90-188-83-170.ngrok-free.app} — адрес, который отобразился в

консоли ngrok. Обратите внимание на протокол. Он должен быть https, иначе Telegram не примет url.

Если получен ответ «ok»: true, то все в порядке.

```
(base) nnyamkin@lxc-jhub01:~/course_project_Yamkin_tpu$ curl --location --request POST 'https://api.telegram.org/bot6611867477:AAHlRyIQPPTdIt5o1x02VFcpRfWdHbgsyyI/setWebhook' \
> --header 'Content-Type: application/json' \
> --data-raw '{"url": "https://25c0-90-188-83-170.ngrok-free.app"}'
{"ok":true,"result":true,"description":"Webhook was set"}(base) nnyamkin@lxc-jhub01:~/course_project_Yamkin_tpu$
```

Рисунок 7 – Результат выполнения команды выше

Теперь всё готово для реализации сервиса на Flask.

Реализация веб-сервиса

Реализуем веб-сервис с помощью фреймворка Flask и Langchain.

Flask — фреймворк для создания веб-приложений на языке программирования Python, использующий набор инструментов Werkzeug, а также шаблонизатор Jinja2. Относится к категории так называемых микрофреймворков — минималистичных каркасов веб-приложений, сознательно предоставляющих лишь самые базовые возможности [6].

Для работы с языковой моделью, нам также потребуется фреймворк Langchain.

Основные компоненты фреймворка:

- **Модели:** универсальный интерфейс для работы с различными языковыми моделями. Можно использовать API OpenAI, Cohere, Hugging Face и других. Также есть возможность работать с локальными моделями [8].
- **Промпты:** LangChain предоставляет ряд функции для работы с промптами — представление промпта согласно типу модели, формирование шаблона на основе внешних данных, форматирование вывода модели [8].
- **Цепочки:** с помощью цепочек можно объединять разные языковые модели и запросы в многоступенчатые конвейеры. Цепочки могут быть применены для разговоров, ответов на вопросы, суммаризаций и других сценариев [8].

- **Агенты:** с помощью агентов модель может получить доступ к различным источникам информации, таким как Google, Wikipedia и т.д [8].

- **Память:** позволяет сохранять состояния в цепочках. Например, для создания чат-бота можно сохранять предыдущие вопросы и ответы. Существует два типа памяти: краткосрочная и долгосрочная. Краткосрочная память передает данные в рамках одного разговора. Долгосрочная память отвечает за доступ и обновление информации между разговорами [8].

В рамках данной работы протестируем работу различных комбинаций указанных выше компонентов фреймворка Langchain с LLM Saiga.

Первым шагом в разработке сервиса импортируем нужные библиотеки.

```
from langchain.llms import LlamaCpp
from langchain.chains import ConversationChain
from langchain.chains.conversation.memory import ConversationBufferMemory
from langchain.agents import initialize_agent
from langchain.utilities import GoogleSearchAPIWrapper
from langchain.tools import Tool
from langchain.prompts import PromptTemplate

from dotenv import load_dotenv
import os

from flask import Flask, request
import requests
```

Затем инициализируем переменные окружения.

```
load_dotenv()
google_api_key = os.getenv("GOOGLE_API_KEY")
google_cse_id = os.getenv("GOOGLE_CSE_ID")
telegram_token = os.getenv("TELEGRAM_TOKEN")
telegram_api = os.getenv("TELEGRAM_API")
```

Укажем путь до большой языковой модели, зададим её параметры и проинициализируем её.

```
model_path = "../privateGPT_latest/privateGPT/models/model-q4_K.gguf"
model_n_ctx = 2048
model_n_batch = 256
# инициализируем LLM
llm = LlamaCpp(model_path=model_path,
               n_ctx=model_n_ctx,
               n_batch=model_n_batch,
```

```
verbose=True,  
n_gpu_layers=40,  
temperature=0.0)
```

Параметр `model_n_ctx` указывает на максимальное количество токенов в контексте, которое может обработать модель.

Параметр `model_n_batch` указывает на количество токенов, которые будут обрабатываться параллельно на GPU.

Параметр `verbose` отвечает за отображение логов модели.

Параметр `n_gpu_layers` определяет количество слоев модели, работающих на GPU.

Параметр `temperature` отвечает за «строгость» ответа. Чем ниже значение температуры, тем более детерминированными будут результаты в смысле того, что будет выбран наиболее вероятный следующий токен. Увеличение температуры может привести к большей случайности, что способствует более разнообразным или творческим результатам.

Память

Для начала научим бота работать с памятью. Память позволяет LLM запоминать предыдущие взаимодействия с пользователем. По умолчанию LLM является `stateless`, что означает независимую обработку каждого входящего запроса от других взаимодействий. На самом деле, когда мы вводим запросы в интерфейсе ChatGPT, под капотом там тоже работает память. В LangChain есть различные реализации памяти, посмотрим, но остановимся на `ConversationBufferMemory`, т.к. это самый простой вид памяти [9].

Создадим цепочку `ConversationChain` и передадим в неё языковую модель и память.

```
conversation = ConversationChain(llm = llm,  
                                memory = ConversationBufferMemory()  
                                )
```

Далее реализуем сервис на Flask.

```
app = Flask(__name__)

@app.route('/', methods = ['POST'])
def send_message():
    chat_id = request.json["message"]["chat"]["id"]

    user_text = request.json["message"]["text"]

    llm_answer = conversation(user_text)['response']
    # llm_answer = google_search_agent.run(user_text)

    # text = 'Привет! Я помогу тебе защитить курсач!'
    data = {'chat_id':chat_id,
           'text':llm_answer}

    method = "sendMessage"
    url = telegram_api + f'/bot{telegram_token}/{method}'
    requests.post(url = url, data = data)
    # print(request.json)
    return {'ok':True}

if __name__ == '__main__':
    app.run(host='0.0.0.0', port = '5015')
```

Протестируем работу LLM в чате Телеграмм бота.

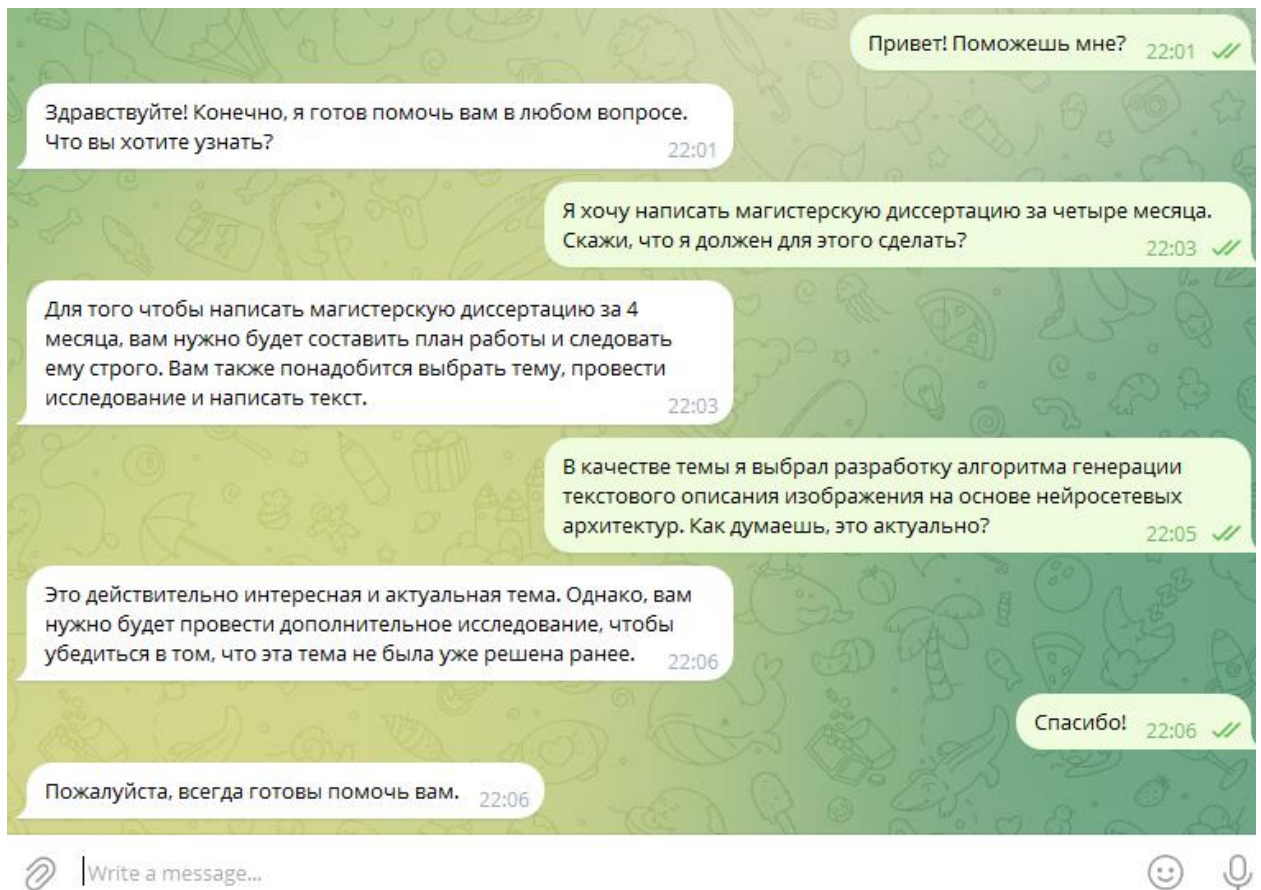


Рисунок 8 – Тестирование работы ConversationBufferMemory

```
[ ] 1 conversation("Мне нужна помощь по учёбе")

Llama.generate: prefix-match hit

llama_print_timings:    load time =   277.57 ms
llama_print_timings:   sample time =    17.62 ms /   33 runs (   0.53 ms per token, 1872.45 tokens per second)
llama_print_timings: prompt eval time =   226.89 ms /   68 tokens (   3.34 ms per token, 299.70 tokens per second)
llama_print_timings:   eval time =   848.85 ms /   32 runs (  26.53 ms per token,  37.70 tokens per second)
llama_print_timings: total time =  1154.74 ms
{'input': 'Мне нужна помощь по учёбе',
 'history': 'Human: Привет! Поможешь мне?\nAI: Здравствуйте! Конечно, я готов помочь вам в любом вопросе. Что вы хотите узнать?',
 'response': 'Какие конкретно темы вас интересуют? Я могу дать вам подробную информацию о каждой из них.'}

[ ] 1 conversation.memory.buffer

'Human: Привет! Поможешь мне?\nAI: Здравствуйте! Конечно, я готов помочь вам в любом вопросе. Что вы хотите узнать?\nHuman: Мне ну
конкретно темы вас интересуют? Я могу дать вам подробную информацию о каждой из них.'
```

Рисунок 9 – Вывод содержимого буфера

Видно, что при использовании буферной памяти, в history просто записываются все предыдущие сообщения.

Такой тип памяти достаточно хорош. Но есть нюансы. Чем длиннее история, тем больше токенов вы будете подавать на вход. Это накладно по финансам, и, даже если вы не бережете свои деньги, возможности модели не безграничны – около 4k токенов.

В Langchain существуют разные виды памяти, помимо ConversationBufferMemory. Вот некоторые из них:

- ConversationSummaryBufferMemory. Этот вид памяти суммаризует историю разговора перед передачей её в параметр history. Таким образом, можно сократить объем текста, обрабатываемого моделью, и улучшить скорость обработки запросов без существенной потери качества.
- ConversationSummaryBufferMemory – позволяет задать окно (в токенах), в рамках которого мы сохраняем диалоги в неизменной форме, а при превышении - суммаризируем.
- ConversationBufferWindowMemory – сохраняет только последние k диалогов.
- ConversationKGMemory - использует граф знаний для сохранения памяти.
- ConversationEntityMemory - сохраняет в памяти знания об определенном объекте [9].

Системный промпт и память

Промпт для языковой модели – это набор инструкций или входных данных, предоставляемых пользователем, которые помогают ей понять контекст и генерировать релевантные и связанные выходные данные.

Создадим собственный системный промпт для модели Saiga и подключим память с помощью цепочки ConversationChain.

```
template = """ Ты – профессиональный маркетолог с опытом написания
высококонверсионной рекламы.
Для генерации рекламного текста ты изучаешь потенциальную целевую
аудиторию и оптимизируешь рекламный текст так,
чтобы он обращался именно к этой целевой аудитории. Напиши рекламный текст
для следующих продуктов/услуг.
Создай текст объявления с привлекающим внимание заголовком и
убедительным призывом к действию, который побуждает пользователей к
целевому действию.

История:{history}
Текст для анализа: {input}
"""

PROMPT = PromptTemplate(input_variables=["history", "input"],
template=template)

conversation = ConversationChain(
    prompt=PROMPT,
    llm=llm,
    verbose=True,
    memory=ConversationBufferMemory(ai_prefix="AI"),
)
```

При этом программный код сервиса Flask останется без изменений.

Протестируем работу LLM в боте.



Рисунок 10 – Работа бота с системным промптом и памятью

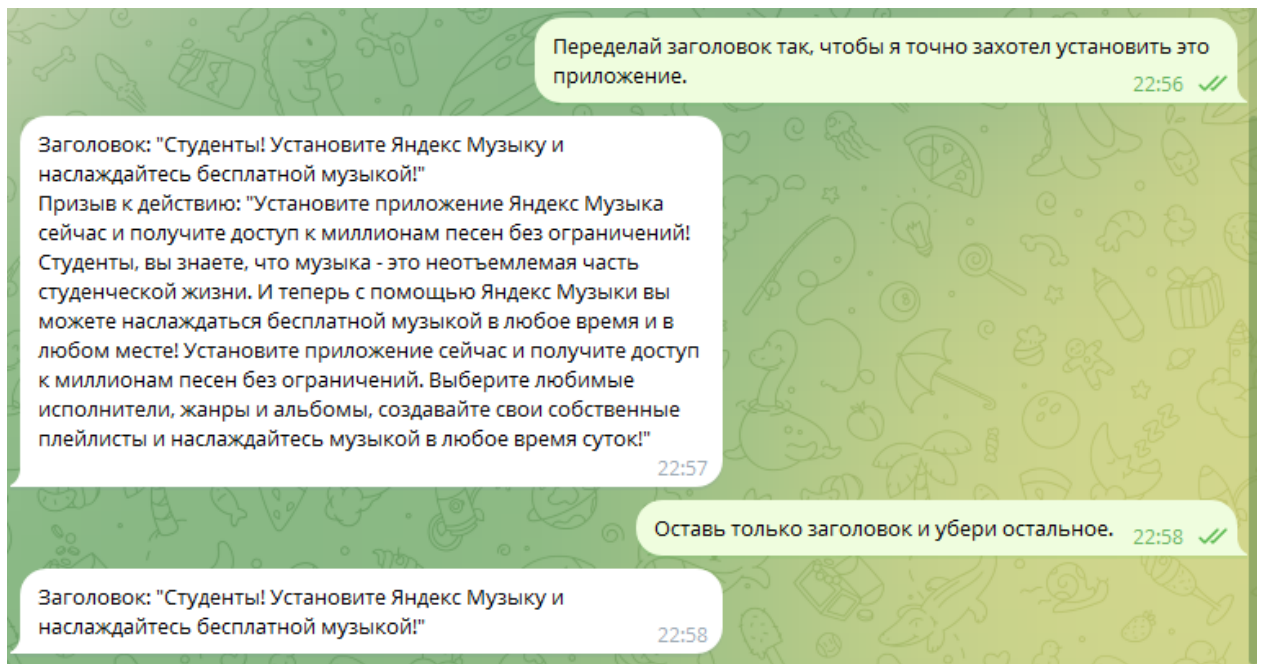


Рисунок 11 – Работа бота с системным промптом и памятью

Видим, что бот отвечает в нужном формате и запоминает контекст диалога.

Агент Google-поиска

Теперь пора переходить к логике взаимодействия с агентами. Ее можно разложить на три компонента:

1. Языковая модель (LLM).
2. Инструмент (Tool) – дополнительный функционал, который вы добавляете к LLM, например калькулятор или поиск. В фреймворке есть уже готовые реализации, но также можно создать свой инструмент.
3. Агент (Agent) – это посредник между LLM и Tool. В одном агенте вы можете использовать сразу несколько инструментов и вызывать их в зависимости от контекста задачи [9].

Известно, что языковые модели не имеют доступа к актуальной информации. Поэтому попробуем создать агента, который добывает для LLM информацию из Google-поиска.

Чтобы работать с поиском, нужно предварительно получить ключ API и зарегистрировать свой движок.

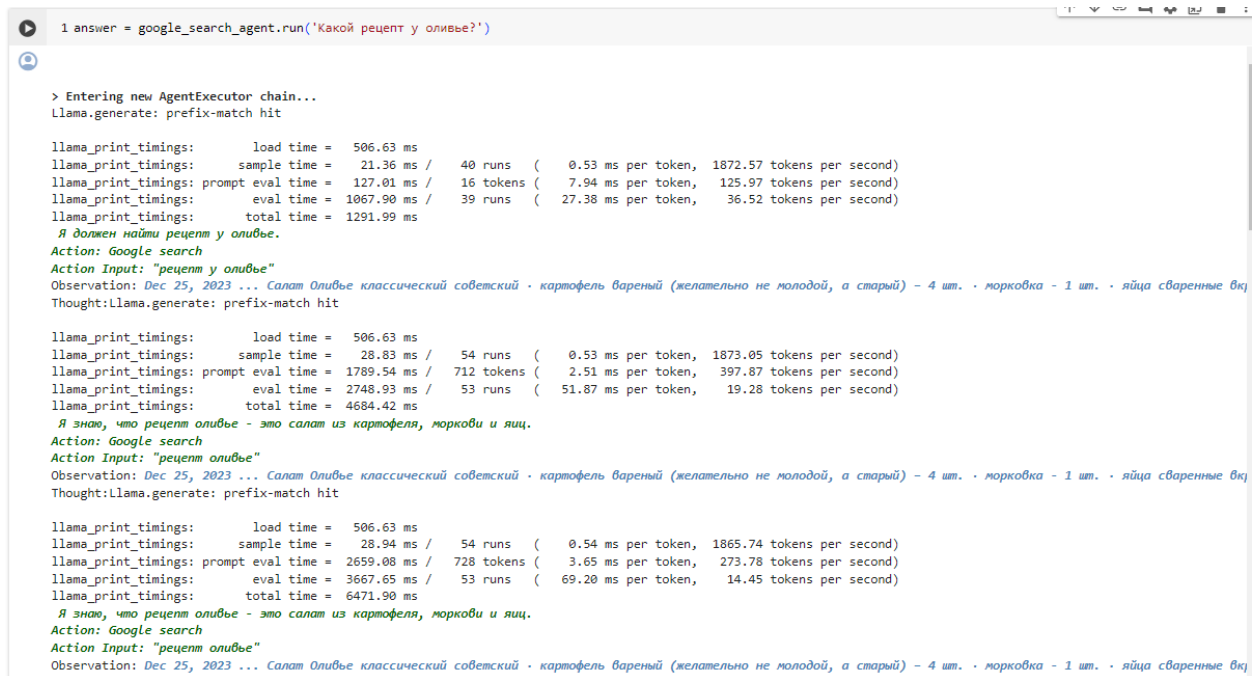
```
# Обертка для вызова API
search = GoogleSearchAPIWrapper()

# инициализируем поисковый инструмент
tool = Tool(name = 'Google search',
            description = 'Search Google for recent results',
            func = search.run)

# инициализируем агента
google_search_agent = initialize_agent(
    agent = 'zero-shot-react-
description',
    tools = [tool],
    llm = llm,
    verbose = True,
    max_iterations = 5)
```

При создании агента, нужно указать его тип, в данном случае мы используем zero-shot-react-description. Но вообще их множество, например: self-ask-with-search, react-docstore, conversational-react-description.

Протестируем работу LLM Saiga с агентом Google-поиска.



```
1 answer = google_search_agent.run('Какой рецепт у оливье?')
```

> Entering new AgentExecutor chain...

Llama.generate: prefix-match hit

llama_print_timings: load time = 506.63 ms
llama_print_timings: sample time = 21.36 ms / 40 runs (0.53 ms per token, 1872.57 tokens per second)
llama_print_timings: prompt eval time = 127.01 ms / 16 tokens (7.94 ms per token, 125.97 tokens per second)
llama_print_timings: eval time = 1067.90 ms / 39 runs (27.38 ms per token, 36.52 tokens per second)
llama_print_timings: total time = 1291.99 ms

Я должен найти рецепт у оливье.

Action: Google search

Action Input: "рецепт у оливье"

Observation: Dec 25, 2023 ... Салат Оливье классический советский · картофель вареный (желательно не молодой, а старый) – 4 шт. · морковь - 1 шт. · яйца сваренные вк

Thought:Llama.generate: prefix-match hit

llama_print_timings: load time = 506.63 ms
llama_print_timings: sample time = 28.83 ms / 54 runs (0.53 ms per token, 1873.05 tokens per second)
llama_print_timings: prompt eval time = 1789.54 ms / 712 tokens (2.51 ms per token, 397.87 tokens per second)
llama_print_timings: eval time = 2748.93 ms / 53 runs (51.87 ms per token, 19.28 tokens per second)
llama_print_timings: total time = 4684.42 ms

Я знаю, что рецепт оливье - это салат из картофеля, моркови и яиц.

Action: Google search

Action Input: "рецепт оливье"

Observation: Dec 25, 2023 ... Салат Оливье классический советский · картофель вареный (желательно не молодой, а старый) – 4 шт. · морковь - 1 шт. · яйца сваренные вк

Thought:Llama.generate: prefix-match hit

llama_print_timings: load time = 506.63 ms
llama_print_timings: sample time = 28.94 ms / 54 runs (0.54 ms per token, 1865.74 tokens per second)
llama_print_timings: prompt eval time = 2659.08 ms / 728 tokens (3.65 ms per token, 273.78 tokens per second)
llama_print_timings: eval time = 3667.65 ms / 53 runs (69.20 ms per token, 14.45 tokens per second)
llama_print_timings: total time = 6471.90 ms

Я знаю, что рецепт оливье - это салат из картофеля, моркови и яиц.

Action: Google search

Action Input: "рецепт оливье"

Observation: Dec 25, 2023 ... Салат Оливье классический советский · картофель вареный (желательно не молодой, а старый) – 4 шт. · морковь - 1 шт. · яйца сваренные вк

Thought:

Рисунок 12 – Работа агента в Google Colab

Посмотрим на шаблон промпта модели при использовании агента Google.

```
[ ] 1 google_search_agent.agent.llm_chain.prompt.template
```

```
'Question: Who lived longer, Muhammad Ali or Alan Turing?\nAre follow up questions needed here: Yes.\nFollow up: How old was Muhammad Ali when he died?\nIntermediate answer: Muhammad Ali was 74 years old when he died.\nFollow up: How old was Alan Turing when he died?\nIntermediate answer: Alan Turing was 41 years old when he died.\nSo the final answer is: Muhammad Ali\n\nQuestion: When was the founder of craigslist born?\nAre follow up questions needed here: Yes.\nFollow up: Who was the founder of craigslist?\nIntermediate answer: Craigslist was founded by Craig Newmark.\nFollow up: When was Craig Newmark born?\nIntermediate answer: Craig Newmark was born on December 6, 1952.\nSo the final answer is: December 6, 1952\n\nQuestion: Who was the maternal grandfather of George Washington?\nAre follow up questions needed here: Yes.\nFollow up: Who was the mother of George Washington?\nIntermediate answer: The mother of George Washington was Mary Ball Washington.\nFollow up: Who was the father of Mary Ball Washington?\nIntermediate answer: The father of Mary Ball Washington was Joseph Ball.\nSo the final answer is: Joseph Ball\n\nQuestion: Are both the directors of Jaws and Casino Royale from the same country?\nAre follow up questions needed here: Yes.\nFollow up: Who is the director of Jaws?\nIntermediate answer: The director of Jaws is Steven Spielberg.\nFollow up: Where is Steven Spielberg from?\nIntermediate answer: The United States.\nFollow up: Who is the director of Casino Royale?\nIntermediate answer: The director of Casino Royale is Martin Campbell.\nFollow up: Where is Martin Campbell from?\nIntermediate answer: New Zealand.\nSo the final answer is: No\n\nQuestion: {input}\nAre followup questions needed here:{agent_scratchpad}'
```

Рисунок 13 – Шаблон промпта модели по умолчанию, при использовании агента

Выглядит он довольно странно. Как оказалось все не случайно, а основано на последних достижениях науки. Такая форма обеспечивает лучшее качество в вопросно-ответных задачах.

Протестируем работу агента в телеграмме.

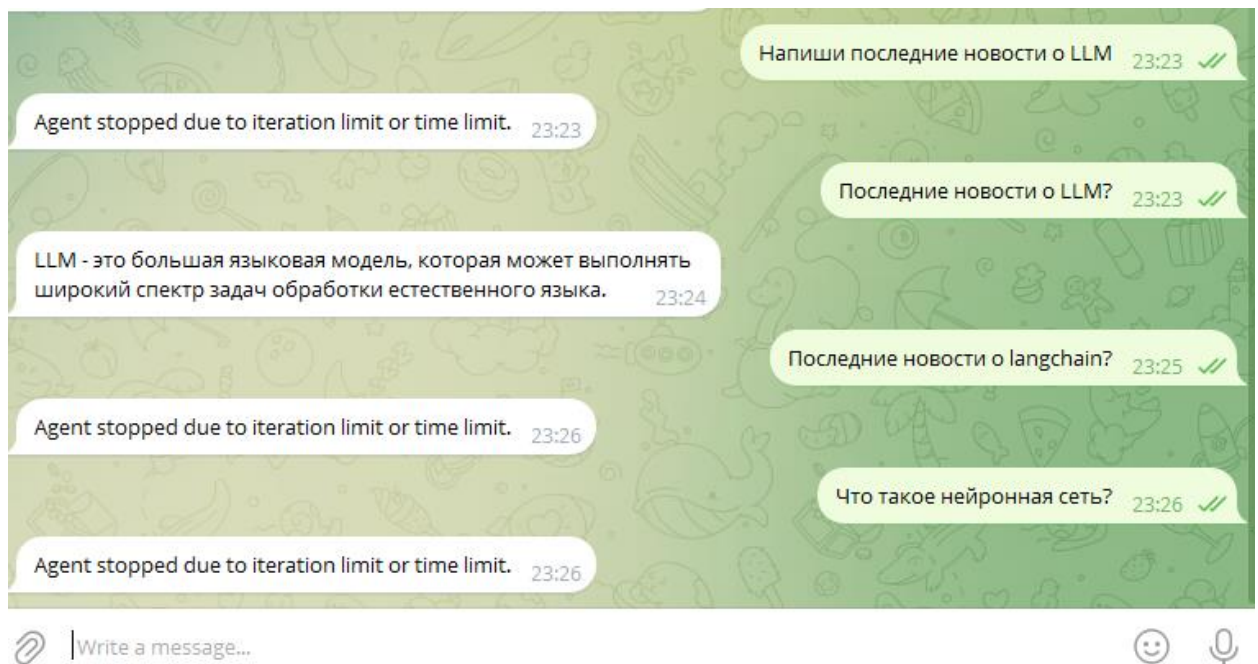


Рисунок 14 – Работа агента в телеграм боте

```

Я должен найти последние новости о LLM.
Action: Google search
Action Input: "LLM"
Observation: Large language model ... A large language model (LLM) is a language model notable for its ability to achieve general-purpose language generation. LLMs acquire ... Learn how the AI algorithm known as a large language model, or LLM, uses deep learning and large data sets to understand and generate new content. What are Large Language Models? Large language models (LLM) are very large deep learning models that are pre-trained on vast amounts of data. The underlying ... An LLM, or Master of Laws, is a graduate qualification in the field of law. The LLM was created for lawyers to expand their knowledge, study a specialized area ... Large language model definition. A large language model (LLM) is a deep learning algorithm that can perform a variety of natural language processing (NLP) tasks ... LLM# ... A CLI utility and Python library for interacting with Large Language Models, both via remote APIs and models that can be installed and run on your own ... Welcome to LSAC LLM and Other Law Programs, formerly known as LSAC's LLM Service. LSAC is expanding to support other types of law programs offered by ABA- ... Your LLM at Penn Carey Law begins with a five-week five-credit Pre-term Program prior to the start of the fall semester. This innovative program provides an ... May 18, 2023 ... Note that using LLMs/embeddings/.. from Julia is barely different than using them from Python. Thanks to PyCall or PythonCall, syntax for calling ... Our Programs - LLM Degree Program - Wharton Business and Law Certificate for LLMs - SJD - Doctor of the Science of Law - LLCM - Master in Comparative Law ...
Thought:
llama.generate: prefix-match hit

llama_print_timings:      load time =   506.63 ms
llama_print_timings:      sample time =    29.47 ms /   55 runs (   0.54 ms per token, 1866.05 tokens per second)
llama_print_timings: prompt eval time =   964.69 ms /  407 tokens (   2.37 ms per token,  421.90 tokens per second)
llama_print_timings:      eval time =  2353.79 ms /   54 runs (  43.59 ms per token,   22.94 tokens per second)
llama_print_timings:      total time =  3460.63 ms

Я знаю, что LLM - это большой языковой модель.
Final Answer: LLM - это большая языковая модель, которая может выполнять широкий спектр задач обработки естественного языка.
> Finished chain.

```

Рисунок 15 – Работа агента в Google Colab

Как видно из скриншотов, агент Google-поиска работает нестабильно. Сайга не генерирует ожидаемый ответ на запрос пользователя. Возможно, это связано с неверной конфигурацией инструмента Tool, либо самого агента.

ЗАКЛЮЧЕНИЕ

Цель работы заключалась в реализации Telegram-бота с использованием русскоязычной LLM Saiga. В ходе выполнения работы были выполнены следующие задачи для достижения этой цели:

1. Были исследованы основные виды больших языковых моделей (BERT-like и GPT-like). Для тестирования работы бота была выбрана русскоязычная генеративная модель Saiga, с 13 миллиардами параметров.

2. Был создан телеграм бот и настроено соединение с локальным сервером.

3. Была реализована логика работы веб-сервера, с помощью фреймворка Flask и Langchain.

4. К Telegram-боту были подключены и протестированы различные инструменты для работы с языковыми моделями, предоставляемые Langchain: буферная память для модели, системный промпт и агент Google-поиска.

СПИСОК ИСТОЧНИКОВ

1. Осваивают ли LLM модели мира, или лишь поверхностную статистику? // Хабр. Свободная энциклопедия URL: <https://habr.com/ru/companies/wunderfund/articles/729532/> (дата обращения: 04.02.2024).

2. Революция AI 2.0. LLM+GPT: история, типы и влияния на мир вокруг нас // Just AI URL: https://just-ai.com/wp-content/uploads/2023/10/Revolution_AI_2-0_part1_2023.pdf (дата обращения: 04.02.2024).

3. Большие языковые модели (LLM) в задачах // Хабр. Свободная энциклопедия URL: <https://habr.com/ru/articles/775870/> (дата обращения: 04.02.2024).

4. Зоопарк трансформеров: большой обзор моделей от BERT до Alpaca // Хабр. Свободная энциклопедия URL: https://habr.com/ru/companies/just_ai/articles/733110/ (дата обращения: 04.02.2024).

5. Saiga2 13B Lora // Hugging Face URL: https://huggingface.co/IlyaGusev/saiga2_13b_lora (дата обращения: 04.02.2024).

6. Прикладной Python: Telegram бот для приема платежей на Flask с нуля // YouTube URL: <https://www.youtube.com/watch?v=GRtgLlwxpc4&t=885s> (дата обращения: 04.02.2024).

7. Простой Telegram-бот на Flask с информированием о погоде // Хабр. Свободная энциклопедия URL: <https://habr.com/ru/articles/495036/> (дата обращения: 04.02.2024).

8. LangChain: создаем свой AI в несколько строк // Хабр. Свободная энциклопедия URL: <https://habr.com/ru/articles/729664/> (дата обращения: 04.02.2024).

9. LangChain для бывалых: память и агенты. часть 1 // Хабр. Свободная энциклопедия URL: <https://habr.com/ru/articles/733262/> (дата обращения: 04.02.2024).